

Actividad de análisis

Mutex en FreeRTOS

Implementacion de soluciones robóticas para las empresas

Juan Gilberto Prado Matehuala

05/10/2026

Objetivo general

- Crear y usar un mutex (xSemaphoreCreateMutex, xSemaphoreTake, xSemaphoreGive).

Objetivos específicos

- Reconocer una condición de carrera sobre un recurso compartido al observarla directamente.
- Diferenciar el propósito de un mutex (exclusión mutua) del de un semáforo binario (Práctica 03:señalización de evento).
- Relacionar el mutex con recursos compartidos reales del proyecto integrador, como un bus de comunicación con varios sensores.

Introducción

El puerto Serial es un recurso único: solo hay uno, y todo lo que se envía a él sale por el mismo cable en el mismo orden. Si dos tareas llaman a Serial.print al mismo tiempo —cada una tratando de imprimir un mensaje de varias partes—, el planificador puede interrumpir a una a la mitad de su mensaje para dejar correr a la otra. El resultado es una salida mezclada: fragmentos de un mensaje intercalados con fragmentos del otro.

Procedimiento

Provocar el problema

Cargar primero el Listado 2, sin protección, y observar el monitor serial: los mensajes de ambas tareas deberían aparecer mezclados o incompletos

```

#include <Arduino.h>

// SemaphoreHandle_t mutex_Serial;

void Tarea_A(void *Parametro) {
    // put your task code here, to run repeatedly:
    for (;;) {
        // xSemaphoreTake(mutex_Serial, portMAX_DELAY);
        Serial.print("distancias para colision-");
        Serial.print("izquierda -");
        Serial.print("derecha -");
        Serial.print("frente -");
        Serial.println("atras -");
        // xSemaphoreGive(mutex_Serial);
        vTaskDelay(pdMS_TO_TICKS(100)); // Delay for 1 second
    }
}

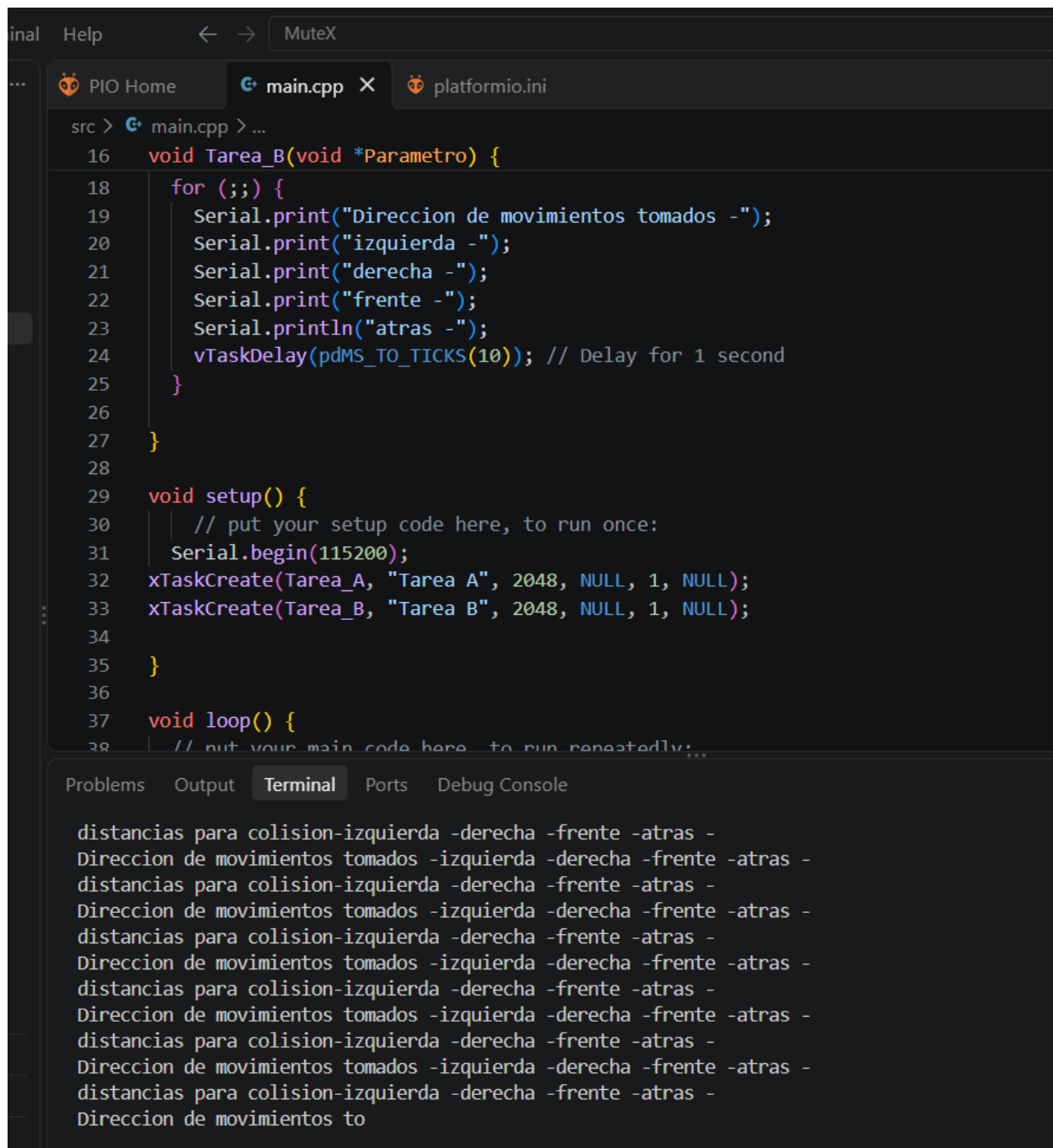
void Tarea_B(void *Parametro) {
    // put your task code here, to run repeatedly:
    for (;;) {
        // xSemaphoreTake(mutex_Serial, portMAX_DELAY);
        Serial.print("Direccion de movimientos tomados -");
        Serial.print("izquierda -");
        Serial.print("derecha -");
        Serial.print("frente -");
        Serial.println("atras -");
        // xSemaphoreGive(mutex_Serial);
        vTaskDelay(pdMS_TO_TICKS(10)); // Delay for 1 second
    }
}

void setup() {
    // put your setup code here, to run once:
    Serial.begin(115200);
}

```

Default (Mute)  Auto

```
void setup() {  
    // put your setup code here, to run once:  
    Serial.begin(115200);  
    // mutex_Serial = xSemaphoreCreateMutex();  
    xTaskCreate(Tarea_A, "Tarea A", 2048, NULL, 1, NULL);  
    xTaskCreate(Tarea_B, "Tarea B", 2048, NULL, 1, NULL);  
  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
}
```



The screenshot shows the Arduino IDE interface. The top menu bar includes 'File', 'Edit', 'Tools', 'Window', and 'Help'. The toolbar has icons for undo, redo, and a search icon. The 'Tools' menu is open, showing 'MuteX'. The 'Window' menu is also open, showing 'main.cpp' and 'platformio.ini'. The main editor displays the code for 'main.cpp' with line numbers 16 to 38. The code defines two tasks, 'Tarea_A' and 'Tarea_B', and a 'loop()' function. The terminal output shows the execution of the code, displaying the direction of movement and the distances for collision.

```
src > main.cpp > ...
16 void Tarea_B(void *Parametro) {
17     for (;;) {
18         Serial.print("Direccion de movimientos tomados -");
19         Serial.print("izquierda -");
20         Serial.print("derecha -");
21         Serial.print("frente -");
22         Serial.println("atras -");
23         vTaskDelay(pdMS_TO_TICKS(10)); // Delay for 1 second
24     }
25 }
26
27 void setup() {
28     // put your setup code here, to run once:
29     Serial.begin(115200);
30     xTaskCreate(Tarea_A, "Tarea A", 2048, NULL, 1, NULL);
31     xTaskCreate(Tarea_B, "Tarea B", 2048, NULL, 1, NULL);
32 }
33
34 void loop() {
35     // put your main code here, to run repeatedly:
36 }
```

Problems Output Terminal Ports Debug Console

distancias para colision-izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
distancias para colision-izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
distancias para colision-izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
distancias para colision-izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
distancias para colision-izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
distancias para colision-izquierda -derecha -frente -atras -
Direccion de movimientos to

Crear el mutex y Proteger la sección crítica

Declarar el mutex como variable global y crearlo en setup():

Envolver el bloque de impresión de cada tarea con xSemaphoreTake/xSemaphoreGive (Listado 4).

```

#include <Arduino.h>

SemaphoreHandle_t mutex_Serial;

void Tarea_A(void *Parametro) {
    // put your task code here, to run repeatedly:
    for (;;) {
        xSemaphoreTake(mutex_Serial, portMAX_DELAY);
        Serial.print("distancias para colision-");
        Serial.print("izquierda -");
        Serial.print("derecha -");
        Serial.print("frente -");
        Serial.println("atras -");
        xSemaphoreGive(mutex_Serial);
        vTaskDelay(pdMS_TO_TICKS(100)); // Delay for 1 second
    }
}

void Tarea_B(void *Parametro) {
    // put your task code here, to run repeatedly:
    for (;;) {
        xSemaphoreTake(mutex_Serial, portMAX_DELAY);
        Serial.print("Direccion de movimientos tomados -");
        Serial.print("izquierda -");
        Serial.print("derecha -");
        Serial.print("frente -");
        Serial.println("atras -");
        xSemaphoreGive(mutex_Serial);
        vTaskDelay(pdMS_TO_TICKS(10)); // Delay for 1 second
    }
}

void setup() {
    // put your setup code here, to run once:
}

```

```
void setup() {  
    // put your setup code here, to run once:  
    Serial.begin(115200);  
    mutex_Serial = xSemaphoreCreateMutex();  
    xTaskCreate(Tarea_A, "Tarea A", 2048, NULL, 1, NULL);  
    xTaskCreate(Tarea_B, "Tarea B", 2048, NULL, 1, NULL);  
  
}  
  
void loop() {  
    // put your main code here, to run repeatedly:  
}
```

```
src > main.cpp > setup()
20 void Tarea_B(void *Parametro) {
21     // put your task code here, to run repeatedly:
22     for (;;) {
23         xSemaphoreTake(mutex_Serial, portMAX_DELAY);
24         Serial.print("Direccion de movimientos tomados -");
25         Serial.print("izquierda -");
26         Serial.print("derecha -");
27         Serial.print("frente -");
28         Serial.println("atras -");
29         xSemaphoreGive(mutex_Serial);
30         vTaskDelay(pdMS_TO_TICKS(10)); // Delay for 1 second
31     }
32 }
33 }
34
35 void setup() {
36     // put your setup code here, to run once:
37     Serial.begin(115200);
38     mutex_Serial = xSemaphoreCreateMutex();
39     xTaskCreate(Tarea_A, "Tarea A", 2048, NULL, 1, NULL);
40     xTaskCreate(Tarea_B, "Tarea B", 2048, NULL, 1, NULL);
41 }
42
```

Problems Output **Terminal** Ports Debug Console

```
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
distancias para colision-izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -atras -
Direccion de movimientos tomados -izquierda -derecha -frente -at
```

Análisis

Comparación antes/después

Compara una captura del monitor serial del Listado 2 (sin mutex) contra una del Listado 4 (con mutex). Describe puntualmente qué cambió.

sin mutex al inicio de la terminal no se alcanza a distinguir cual de las 2 quiere imprimir primero, por lo que se ve uno puesto sobre otro, seguido de esto ya se imprime uno de los dos, con mutex ya toma un poco mas de orden por lo que se ve mas limpio y con orden, dando prioridad a quien lo requiere.

¿Por qué no un semáforo binario?

Explica por qué usar el mismo semáforo binario de la Práctica 03 —creado con `xSemaphoreCreateBinary`— para este problema sería una mala práctica, aunque técnicamente compile y funcione la mayor parte del tiempo. (Pista: piensa en la herencia de prioridad de la Sección 2.2.)

Un semáforo binario no da prioridad a quien realmente lo necesita, por lo que es mas útil el uso de mutex en cuestión de prioridades para ciertas lecturas que lo necesitan.

Puente al proyecto propio

Identifica un recurso compartido real en el robot de tu equipo que necesitaría un mutex —por ejemplo, un bus I2C con varios sensores, o un mismo motor controlado desde dos tareas distintas. Describe qué pasaría si dos tareas lo usaran al mismo tiempo sin protección.

Cuando se mida velocidad y distancia para una colisión, dándole mas importancia a la colisión silenciando la velocidad ya que es mas importante detener al robot antes de que colisione y surja algún daño o accidente.

PIO Home main.cpp X platformio.ini

src > main.cpp > Distancia_colision(void *)

```
21 void Distancia_colision(void *Parametro) {
22     for (;;) {
23         int distancia = analogRead(Pot_Dir);
24         Serial.print("Distancia ante una colision : ");
25         Serial.println(distancia);
26         vTaskDelay(pdMS_TO_TICKS(1000)); // Delay for 1 second
27     }
28 }
29
30 }
31
32 void setup() {
33     // put your setup code here, to run once:
34     Serial.begin(115200);
35     mutex_Serial = xSemaphoreCreateMutex();
36     xTaskCreate(Leer_velocidad, "Leer velocidad", 2048, NULL, 1, NULL);
37     xTaskCreate(Distancia_colision, "Distancia para colision", 2048, NULL, 1, NULL);
38 }
39
40
41 void loop() {
42     // put your main code here, to run repeatedly:
43 }
```

Problems Output Terminal Ports Debug Console

```
velocidad : 4095
velocidad : 4095
velocidad : 4095
velocidad : 4095
velocidad : 4095
Distancia ante una colision : 4095
velocidad : 4095
velocidad : 4095
velocidad : 4095
velocidad : 4095
velocidad : 4095
v
```